

目录

- 1. 查找与优化 (Search & Optimization)
 - 1.1 整数二分 (Binary Search Int)
 - 1.2 浮点数二分 (Binary Search Float)
 - 1.3 三分搜索 (Ternary Search)
- 2. 区间操作与差分 (Diff & Prefix Sum)
 - 2.1 一维/二维差分 (1D/2D Diff)
 - 2.2 二级差分 (Second-Order Diff)
 - 2.3 树上差分 (Tree Diff)
- 3. 双指针 (Two Pointers)
 - 3.1 对撞指针/滑动窗口 (Collision/Window)
 - 3.2 快慢指针 (Fast/Slow Pointers)
- 4. 数据结构 (Data Structures)
 - 4.1 单调栈/队列 (Monotonic Stack/Queue)
 - 4.2 并查集 (Disjoint Set)
 - 4.3 带权并查集 (Weighted DSU)
- 5. 搜索与图论 (Search & Graph)
 - 5.1 深度/广度优先搜索 (DFS/BFS)
 - 5.2 拓扑排序 (Topological Sort)
 - 5.3 最短路 (Shortest Path: Dijkstra/Floyd/SPFA)
- 6. 动态规划 (DP)
 - 6.1 背包 DP (Knapsack DP)
 - 6.2 线性 DP (Linear DP: LIS)

二分

- 适合答案有单调性

整数二分

```
//这个是找左边界的
// 查找区间 [l, r]
while (l < r) {
    long long mid = (l + r) >> 1; // 向下取整, 不需要 +1
    if (check(mid)) {
        r = mid; // 答案在左边 (包含 mid)
    } else {
        l = mid + 1; // 答案在右边
    }
}
```

```

    }
}
// 循环结束时 l == r, 即为答案
return l;

```

浮点数二分

```

for(int i=0;i<=100;i++)
{
    double mid = (l + r) / 2;
    if (check(mid, jd))
        r = mid;
    else
        l = mid;
}
cout << fixed << setprecision(10) << l << '\n';

```

三分

- 答案有凸函数的性质

整数三分

```

//求极小值
//求极大值: if (f(m1) > f(m2)) r = m2; else l = m1;
//极大值缩点: if (f(i) > f(ans)) ans = i;
long long integer_ternary_search(long long l, long long r) {
    // 1. 先三分缩小范围, 直到区间长度小于 3 (r - l <= 2)
    // 这样保证 m1 和 m2 始终不会重合, 且区间稳定缩小
    while (r - l > 2) {
        long long m1 = l + (r - l) / 3;
        long long m2 = r - (r - l) / 3;

        // 求【极小值】逻辑:
        // 如果左边更小, 说明谷底在左边, 舍弃右边
        if (f(m1) < f(m2)) {
            r = m2;
        } else {
            l = m1;
        }
    }

    // 2. 最后剩下的区间 [l, r] 长度很小 (1~3个点), 直接暴力找最小值
    long long ans = l; // 假设 l 是答案
    for (long long i = l + 1; i <= r; i++) {
        if (f(i) < f(ans)) { // 这一步同样根据求极小/极大调整符号

```

```

        ans = i;
    }
}
return ans;
}

```

浮点数三分

```

// 浮点数三分模板
// 传入：搜索范围 [l, r]
// 返回：极值点的横坐标（如果求纵坐标，外面再套一个 f() 即可）
double ternary_search(double l, double r) {
    // 100次循环足够保证精度，一般题目60-100次均可
    for (int i = 0; i < 100; i++) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;

        // 注意：这里决定了是求极大值还是极小值
        // 当前逻辑：f(m1) < f(m2) 时，说明左边更小，且我们想找更小的 -> 舍弃右边
        // 所以这是【求极小值】
        if (f(m1) < f(m2)) {
            r = m2;
        } else {
            l = m1;
        }

        // 如果要求【极大值】，只需改为：
        // if (f(m1) < f(m2)) l = m1; else r = m2;
    }
    return l; // 最后 l 和 r 极度接近，返回 l 或 r 或 (l+r)/2 都可以
}

```

前缀和/差分

- 区间修改，单点查询
- 注意区间是左闭右开还是左闭右闭
- 差分和前缀和就像导数和积分一样，使用的时候你可以用差分做到线性增长(详情见题)

一维

```

//差分
D[l] += v;
D[r + 1] -= v;
//前缀和
// 此时 D[i] 已经变成了原数组 A[i]
for(int i = 1; i <= n; i++) {

```

```

    D[i] += D[i - 1];
}

```

二维

```

D[x1][y1]      += v;
D[x1][y2 + 1] -= v;
D[x2 + 1][y1] -= v;
D[x2 + 1][y2 + 1] += v;

//前缀和还原
// 假设 D 数组初始化好, 且从 (1,1) 开始
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= m; j++) {
        // 注意: 这里不仅是累加自己, 还要复原二维前缀和公式
        // 现在的 D[i][j] 存的是差分值
        // 还原成原数组 A[i][j]:
        D[i][j] += D[i-1][j] + D[i][j-1] - D[i-1][j-1];
    }
}

```

二级差分 (实现等差数列)

```

void add_AP(int l, int r, int s, int d) {
    // 1. 也就是 D2[1] += s
    // 原因: s 是首项, 影响 D1[1] 的突变, 进而影响 D2[1]
    D2[1] += s;

    // 2. 也就是 D2[l+1] += d - s
    // 原因: 从 l+1 开始进入等差增长, D1 应该变成常数 d,
    // 但 D1[l] 突变了 s, 所以这里要修正。
    D2[l + 1] += d - s;

    // 3. 也就是 D2[r+1] -= s + (r - l + 1) * d
    // 原因: 这是最难理解的。
    // 这里减去的是 (末项 + 公差), 为了抵消掉 D1 在 r+1 处的正向突变
    D2[r + 1] -= s + (long long)(r - l + 1) * d;

    // 4. 也就是 D2[r+2] += s + (r - l) * d
    // 原因: 加回 (末项), 用于平复 r+2 及以后的波动
    D2[r + 2] += s + (long long)(r - l) * d;
}

//还原
// 1. 计算 D1
for(int i = 1; i <= n; i++) D2[i] += D2[i-1]; // 此时 D2 变成了 D1
// 2. 计算 A
for(int i = 1; i <= n; i++) D2[i] += D2[i-1]; // 此时 D2 变成了 A

```

xor差分

- $O(1)$ 查找区间xor和/某个数字
- "区间翻转" 或者 "01状态切换"

```
D[l] ^= v;
D[r + 1] ^= v;
```

乘法差分

```
D[l] = D[l] * v % MOD;
D[r + 1] = D[r + 1] * inv(v) % MOD; // 这里需要用到你的逆元板子
```

离散化差分

```
// 离散化差分标准流程
vector<int> nums; // 存所有出现过的 l 和 r+1
vector<tuple<int, int, int>> ops; // 存操作

// 1. 读入操作, 把 l 和 r+1 扔进 nums
for(...) {
    cin >> l >> r >> v;
    ops.emplace_back(l, r, v);
    nums.push_back(l);
    nums.push_back(r + 1); // 注意是 r+1
}

// 2. 离散化去重排序
sort(all(nums));
nums.erase(unique(all(nums)), nums.end());

// 3. 查找函数
auto get_id = [&](int x) {
    return lower_bound(all(nums), x) - nums.begin() + 1;
};

// 4. 在离散化后的数组上做差分
vector<long long> diff(nums.size() + 5, 0);
for(auto [l, r, v] : ops) {
    diff[get_id(l)] += v;
    diff[get_id(r + 1)] -= v;
}

// 5. 还原时注意:
// 离散化后的 diff[i] 代表的是原数轴上区间 [nums[i-1], nums[i]-1] 的值
```

树上差分 (因为我不会所以以下是AI教程)

```
#include <bits/stdc++.h>
using namespace std;

// ===== 参数设置 (根据题目修改) =====
const int MAXN = 200005; // 最大节点数
const int LOGN = 20;     // log2(MAXN), 一般 20 就够用了

// ===== 全局变量 =====
vector<int> adj[MAXN]; // 邻接表存图
int fa[MAXN][LOGN];   // 倍增数组: fa[u][i] 表示 u 的第 2^i 个祖先
int depth[MAXN];       // 深度数组
long long diff[MAXN];  // 差分数组 (记得开 long long 防止爆)
long long ans[MAXN];   // 最终结果数组
int n, m;

// ===== 第一步: LCA 预处理 =====
// 功能: 计算深度、初始化倍增数组
// 调用方式: dfs_lca(root, 0, 1); (root通常是1)
void dfs_lca(int u, int p, int d) {
    depth[u] = d;
    fa[u][0] = p;
    // 初始化倍增表 (2^1, 2^2 ...)
    for (int i = 1; i < LOGN; i++) {
        fa[u][i] = fa[fa[u][i - 1]][i - 1];
    }

    for (int v : adj[u]) {
        if (v != p) {
            dfs_lca(v, u, d + 1);
        }
    }
}

// 功能: 查询 u 和 v 的最近公共祖先
int get_lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v); // 保证 u 更深

    // 1. 把 u 跳到和 v 同一层
    for (int i = LOGN - 1; i >= 0; i--) {
        if (depth[u] - (1 << i) >= depth[v]) {
            u = fa[u][i];
        }
    }

    if (u == v) return u; // 如果跳到了同一个点, 说明那个点就是 LCA

    // 2. u 和 v 一起向上跳, 跳到 LCA 的下面一层
    for (int i = LOGN - 1; i >= 0; i--) {
```

```

        if (fa[u][i] != fa[v][i]) {
            u = fa[u][i];
            v = fa[v][i];
        }
    }

    return fa[u][0]; // 再往上一步就是 LCA
}

// ===== 第二步：差分操作（二选一） =====

// 【情况A：点差分】 u到v路径上所有点权 +w
void update_node(int u, int v, int w) {
    int lca = get_lca(u, v);
    diff[u] += w;
    diff[v] += w;
    diff[lca] -= w;
    if (fa[lca][0] != 0) { // 防止根节点没有父节点导致越界
        diff[fa[lca][0]] -= w;
    }
}

// 【情况B：边差分】 u到v路径上所有边权 +w
// 注意：点 i 的值代表 "i 到 fa[i]" 这条边
void update_edge(int u, int v, int w) {
    int lca = get_lca(u, v);
    diff[u] += w;
    diff[v] += w;
    diff[lca] -= 2 * w;
}

// ===== 第三步：统计答案 =====
// 功能：自底向上累加，把差分值还原成真实值
// 调用方式：dfs_calc(root, 0);
void dfs_calc(int u, int p) {
    for (int v : adj[u]) {
        if (v != p) {
            dfs_calc(v, u); // 先递归子树
            diff[u] += diff[v]; // 回溯时累加子节点的值
        }
    }
    ans[u] = diff[u]; // 此时 diff[u] 就是最终的增加量
}

// ===== 示例 Main =====

int main() {
    // 1. 读入和建图
    cin >> n >> m;
    for (int i = 1; i < n; i++) {
        int u, v; cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
}

```

```

// 2. 必须先跑一遍 LCA 预处理!!
dfs_lca(1, 0, 1);

// 3. 处理所有修改操作
while (m--) {
    int u, v, w;
    cin >> u >> v >> w;
    // 根据题目要求选一个:
    update_node(u, v, w); // 改点
    // update_edge(u, v, w); // 改边
}

// 4. 还原真实值
dfs_calc(1, 0);

// 5. 输出答案
// 如果是边差分, ans[1] (根节点) 应该是 0
for (int i = 1; i <= n; i++) {
    cout << ans[i] << " ";
}

return 0;
}

```

现场操作手册 (一定要看!) 如果你在考场上遇到了, 按以下步骤思考:

- 判断题型: 题目是不是说“给 m 条路径 (u, v) , 让路径上的都加 k , 最后问结果”? 是就是树上差分
- 区分点/边: 题目说“修改城市的繁荣度” -> 点差分
- 题目说“修改道路的修缮次数” -> 边差分。
- 抄模板: 把上面的 adj, fa, depth, diff, dfs_lca, get_lca, dfs_calc 全部抄上去。
- 改数组大小: 看一眼题目的 N 是多少, 把 MAXN 改得比 N 大一点。
- 套公式: 点差分: u 加, v 加, lca 减, lca 的爸爸减。边差分: u 加, v 加, lca 减双倍。
- 特别提醒: LCA 必须先初始化! 在读入 m 次询问之前, 一定要先调用 `dfs_lca(1, 0, 1)`。
- 最后必须跑 DFS! 所有的修改只是打标记, 最后的 `dfs_calc` 才是真正的计算。

双指针

对撞指针

- 线性结构: 数组、字符串。
- 有序 (或可排序): 题目给的是增序数组, 或者你可以先 `std::sort` 一遍而不影响结果。
- 查找关系: 题目让你找两个元素, 满足某种计算结果 (和、差、积、面积)。
- 两个数满足某种数值关系

```

// 假设 vector<int> nums 是有序的
int l = 0, r = nums.size() - 1;

```

```
while (l < r) { // 注意：如果是查找两数，通常  $l < r$ ；如果是回文串，可能需要  $l \leq r$ 
    int sum = nums[l] + nums[r];
    if (sum == target) {
        // 找到了
        cout << l << " " << r << endl;
        break;
    } else if (sum < target) {
        l++; // 和太小了，左指针右移（让和变大）
    } else {
        r--; // 和太大了，右指针左移（让和变小）
    }
}
```

滑动窗口

- 适用场景：子数组/子串问题（如“满足条件的最长/最短子串”、“连续子数组的和”）。这是ACM中最常考的双指针类型。核心逻辑：两个指针 l 和 r 都从左边出发。 r 主动向右扩展窗口， l 被动向右收缩窗口。

```
// 这是一个通用的滑动窗口模板
// n 是数组长度
for (int r = 0, l = 0; r < n; ++r) {
    // 1. 【进窗】：将 nums[r] 加入窗口，更新状态
    add(nums[r]);

    // 2. 【出窗】：当窗口内的状态不满足条件时 (invalid)，l 指针右移
    while (/* window needs shrink */) {
        remove(nums[l]); // 移除 nums[l] 带来的影响
        l++;             // 左边界收缩
    }

    // 3. 【更新答案】：此时窗口 [l, r] 是合法的
    ans = max(ans, r - l + 1);
}
```

快慢指针

- 适用场景：原地处理数组（去重、移动零）、链表找环、链表找中点。核心逻辑：fast 指针负责探路（遍历所有元素），slow 指针负责构建有效结果（或作为锚点）。

```
int slow = 0;
for (int fast = 0; fast < nums.size(); ++fast) {
    if (nums[fast] != val) {
        // 遇到需要的元素，就把它扔给 slow 指针的位置
        nums[slow] = nums[fast];
        slow++;
    }
    // 如果等于 val, fast 直接跳过, slow 不动
}
```

```
return slow; // slow 即为新数组的长度
```

位运算(详情请见位运算)

数据结构

单调栈/单调队列

- **运用场景**
 - 下一个/上一个更大/更小值的位置
 - 去除重复子串
 - “矩形面积最大”（直方图）。
- **精妙之处** 在栈里面存下标，只在更新的时候“取下标”。
- **AC代码**

```
int n;  
cin >> n;  
vector<int> a(n);  
for (int i = 0; i < n; i++)  
{  
    cin >> a[i];  
}  
  
stack<int> stk;  
vector<int> ans(n);  
for (int i = 0; i < n; i++)  
{  
    while (!stk.empty() && a[stk.top()] < a[i])  
    {  
        ans[stk.top()] = i + 1;  
        stk.pop();  
    }  
    stk.push(i);  
}  
  
for (int i = 0; i < n; i++)  
{  
    cout << ans[i] << " \n"[i == n - 1];  
}
```

单调队列

- **运用场景**
 - 滑动窗口维护区间最大值/最小值
 - 固定窗口的极值
 - “滑动窗口”、“区间长度固定为K”、“连续子序列的最值”

• AC代码

```
int n, k;
cin >> n >> k;
vector<int> a(n + 1);
for (int i = 0; i < n; i++)
{
    cin >> a[i];
}
deque<int> dq;
vi ans1(n + 1);
for (int i = k - 1; i < n; i++)
{
    while (!dq.empty() && dq.front() + k <= i)
    {
        dq.pop_front();
    }
    while (!dq.empty() && a[dq.back()] > a[i])
    {
        dq.pop_back();
    }
    dq.push_back(i);
    ans1[i] = a[dq.front()];
}
for (int i = k - 1; i < n; i++)
{
    cout << ans1[i] << " \n"[i == n - 1];
}
```

并查集

- 关键词：“连通性”、“是否在同一个集合”、“合并集合”、“朋友圈”、“亲戚关系”。
- 典型模型：给你一堆点和边，问你某两点通不通？问你图里分成了几块？（连通分量个数）。
- 警铃：只要题目暗示“不断加边，查询两点关系”，99% 是并查集。
- 能做：极速判断连通性、合并集合（时间复杂度近乎 $O(1)$ ）。
- 不能做：拆分集合（一旦合并，很难拆开）、查询两点之间的具体路径（它只知道连通，不知道怎么走）

```
const int MAXN = 200005; // 根据题目修改
int fa[MAXN]; // father数组，存每个人的老大

// 初始化：每个人最开始的老大是自己
void init(int n) {
    for (int i = 1; i <= n; i++) {
```

```

        fa[i] = i;
    }
}

// 查：找祖宗（带路径压缩，核心！）
// 递归把沿途所有人的老大直接改成祖宗，下次查就是O(1)
int find(int x) {
    if (x == fa[x]) return x;
    return fa[x] = find(fa[x]);
}

// 并：把x和y所在的两个帮派合并
void join(int x, int y) {
    int rootX = find(x);
    int rootY = find(y);
    if (rootX != rootY) {
        fa[rootX] = rootY; // 让x的老大认y的老大做大哥
    }
}

// 判断：x和y是不是一伙的
bool isSame(int x, int y) {
    return find(x) == find(y);
}

```

带权并查集

- **相对数值关系**：题目告诉你：A 比 B 贵 10 元，B 比 C 贵 5 元。询问：A 比 C 贵多少？（这是最简单的线性权值）
- **奇偶性/异或关系**：题目告诉你：A 和 B 奇偶性相同，B 和 C 奇偶性不同。询问：A 和 C 奇偶性相同吗？（这是模 2 的权值）循
- ****环克制关系（石头剪刀布/食物链）**：**题目告诉你：A 吃 B，B 吃 C，C 吃 A。询问：A 和 D 是什么关系？（这是模 3 的权值，经典的POJ 1182 食物链）
- **向量方向要搞清：顺着箭头是加，逆着箭头是减**

```

const int MAXN = 200005;
int fa[MAXN];
int d[MAXN]; // d[x] 表示 x 到 fa[x] 的权值关系（比如距离、差值）

void init(int n) {
    for (int i = 1; i <= n; i++) {
        fa[i] = i;
        d[i] = 0; // 自己到自己距离为0
    }
}

// 带权值的 Find：路径压缩 + 权值更新
int find(int x) {
    if (x != fa[x]) {
        int originFa = fa[x]; // 先记下旧爸爸是谁

```

```

    fa[x] = find(fa[x]);    // 递归找祖宗，并进行路径压缩

    // 核心逻辑：更新 d[x]
    // 现在的 x 到 祖宗 的距离 = x 到 旧爸爸 的距离 + 旧爸爸 到 祖宗 的距离
    d[x] += d[originFa];
}
return fa[x];
}

// 带权值的 Join: 已知 x 和 y 的关系是 val (即 x 到 y 的距离/差值是 val)
// 目标: 把 x 的祖宗(rootX) 接到 y 的祖宗(rootY) 下面
void join(int x, int y, int val) {
    int rootX = find(x);
    int rootY = find(y);
    if (rootX != rootY) {
        fa[rootX] = rootY;

        // 核心推导 (向量运算) :
        // d[rootX] + d[x] = d[y] + val
        // 所以 -> d[rootX] = d[y] + val - d[x]
        d[rootX] = d[y] + val - d[x];
    }
}

// 查 x 和 y 的关系
// 如果 isSame(x, y) 为真, 那么 x 和 y 的相对距离就是 d[x] - d[y]

```

搜索

DFS

- **是否需要恢复状态**: 标记的状态是属于“当前这条路径”的, 还是属于“这个节点”的?
 - 需要恢复现场 (Backtracking) 场景: 寻找所有解、所有路径、全排列、组合、棋盘摆放 (N皇后)。核心逻辑: 一个点可以在不同的路径中被重复利用。虽然在这条路径里我也许不能回头, 但我在尝试下一条路径时, 这个点必须是“干净”的。全排列 / 组合: 数字 1 在第一位用过了, 撤销后, 在第二位还能用。求两点间所有路径: 这条路走不通或者走完了, 退回来, 换个方向走, 刚刚走过的点对于新路径来说是没走过的。
 - 不需要恢复现场 (No Backtracking) 场景: 连通块计数 (Flood Fill)、拓扑排序、求可达性 (能不能到)、图的遍历。核心逻辑: 只要我来过这个点一次, 这个点的任务就完成了 (比如已经被染过色了, 或者已经证明从这出发能到达终点了)。下次再遇到它, 直接跳过, 绝不重复计算。连通块: 这块陆地我已经踩过了, 它属于第 1 个岛, 下次再从别的地方撞到它, 它还是属于第 1 个岛, 不用把脚印擦掉。树的遍历: 树没有环, 只要向下走就不可能回到父节点, 天然不需要判重, 更不需要回溯标记
 - 特例提示: 如果是 记忆化搜索 (Memoization) (比如求最长路), 虽然是在找路径, 但因为我们记录了 $dp[u]$ 的值, 所以也不需要物理上的“恢复 st”, 因为 dp 值本身就代表了该节点的最优解, 不需要重新算。

全排列

```

int path[N]; // 记录路径
bool st[N]; // 记录数字是否被使用过

void dfs(int u) {
    if (u > n) {
        // 输出 path[1...n]
        return;
    }

    for (int i = 1; i <= n; i++) {
        if (!st[i]) {
            path[u] = i;
            st[i] = true; // 标记
            dfs(u + 1);
            st[i] = false; // 回溯: 恢复现场
        }
    }
}
// 调用: dfs(1);

```

组合型枚举

```

// u: 当前枚举到了第几个坑位
// start: 当前只能从哪个数开始选 (避免重复)
void dfs(int u, int start) {
    if (u > k) {
        // 输出 path[1...k]
        return;
    }

    for (int i = start; i <= n; i++) {
        path[u] = i;
        dfs(u + 1, i + 1); // 下一层从 i+1 开始
        // 组合一般不需要显式 st[] 数组回溯, 因为 i+1 保证了不重复
    }
}
// 调用: dfs(1, 1);

```

连通块

```

bool st[N][N]; // 访问标记
int dx[] = {-1, 0, 1, 0}, dy[] = {0, 1, 0, -1};

void dfs(int x, int y) {
    st[x][y] = true;
    for (int i = 0; i < 4; i++) {
        int a = x + dx[i], b = y + dy[i];
        // 越界、障碍、已访问检查
    }
}

```

```

        if (a < 0 || a >= n || b < 0 || b >= m) continue;
        if (g[a][b] != '0' && !st[a][b]) { // 假设 '0' 是水, 非 '0' 是陆地
            dfs(a, b);
        }
    }
}

// 主循环
int cnt = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        if (g[i][j] != '0' && !st[i][j]) {
            dfs(i, j); // 把这一个连通块全染成 true
            cnt++;
        }

```

```

//普通图
bool st[N];
vector<int> g[N];

void dfs(int u) {
    st[u] = true;
    for (int v : g[u]) {
        if (!st[v]) dfs(v);
    }
}

// 主循环
int cnt = 0;
for (int i = 1; i <= n; i++) {
    if (!st[i]) {
        dfs(i);
        cnt++;
    }
}

```

BFS

走迷宫

```

struct Point { int x, y; };
int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};
int dist[N][N]; // 初始化为 -1 或 INF 表示未访问
char g[N][N]; // 地图

int bfs(Point start, Point end) {
    memset(dist, -1, sizeof(dist));
    queue<Point> q;

```

```

q.push(start);
dist[start.x][start.y] = 0;

while (!q.empty()) {
    auto t = q.front(); q.pop();

    if (t.x == end.x && t.y == end.y) return dist[t.x][t.y];

    for (int i = 0; i < 4; i++) {
        int a = t.x + dx[i], b = t.y + dy[i];
        // 越界 check、障碍物 check、是否已访问 check
        if (a >= 0 && a < n && b >= 0 && b < m && g[a][b] != '#' && dist[a][b]
== -1) {
            dist[a][b] = dist[t.x][t.y] + 1;
            q.push({a, b});
        }
    }
}
return -1; // 无法到达
}

```

拓扑排序

- 适用于有向无环图 (DAG) 的排序或判环。

```

int in[N];           // 入度
vector<int> g[N];     // 邻接表
vector<int> ans;      // 存储结果序列

bool toposort() {
    queue<int> q;
    // 1. 入度为0的点入队
    for (int i = 1; i <= n; i++) {
        if (in[i] == 0) q.push(i);
    }

    while (!q.empty()) {
        int u = q.front(); q.pop();
        ans.push_back(u);

        for (int v : g[u]) {
            if (--in[v] == 0) { // 2. 删边, 若入度变为0则入队
                q.push(v);
            }
        }
    }
    // 如果 ans.size() == n, 说明是 DAG; 否则存在环
    return ans.size() == n;
}

```

最短路

Dijkstra 非负权最短路

- AC 代码:

```
void solve()
{
    ll n,m,s;
    cin>>n>>m>>s;
    //前面是点,后面是距离
    vector<vector<pair<int,ll>>> mp(n+1);
    rep(i,0,m-1)
    {
        int u,v,w;
        cin>>u>>v>>w;
        mp[u].push_back({v,w});
        // mp[v].push_back({u,w});这道题是有向边所以不要入两次!!
    }

    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>>pos ;
    vector<ll> dist(n+1,LINF);
    dist[s]=0;
    pos.emplace(0,s);

    while(!pos.empty())
    {
        auto [nowd,nowp] = pos.top();
        pos.pop();
        if(nowd>dist[nowp])continue;
        for(auto [nextp,nextd]:mp[nowp])
        {
            if(dist[nextp]>dist[nowp]+nextd)
            {
                dist[nextp]=dist[nowp]+nextd;
                pos.emplace(dist[nextp],nextp);
            }
        }
    }
    rep(i,1,n)
    {
        cout<<dist[i]<<' ';
    }
}
```

- 注意事项:

- 注意Dijkstra算法用最小堆优化可以时间复杂度最低
- Dijkstra算法只能处理**非负权路径问题**
- 为什么不用队列? : 贪心最快, 如果是菊花图复杂度会退化到nm

- 含负权路用什么算法？用队列

Floyd-Warshall多源最短路径 (All-Pairs Shortest Path) $O(n^3)$

```
#include <algorithm>
#include <cstring>

using namespace std;

const int N = 405; // Floyd一般用于 N <= 400 的情况
const int INF = 0x3f3f3f3f; // 无穷大, 防止加法溢出

int d[N][N]; // 邻接矩阵, d[i][j] 表示从 i 到 j 的最短距离
int n, m;    // 点数, 边数

void floyd() {
    // 核心逻辑: k 必须在最外层!
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                // 松弛操作
                // 只有当 k 能够优化 i->j 的路径时才更新
                // 判断 d[i][k] 和 d[k][j] != INF 是为了防止无穷大相加溢出 (如果INF设
                // 得很大)
                if (d[i][k] != INF && d[k][j] != INF) {
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
                }
            }
        }
    }
}

int main() {
    // 1. 初始化
    // 自己到自己距离为0, 到其他人默认为无穷大
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) {
            if (i == j) d[i][j] = 0;
            else d[i][j] = INF;
        }
    }

    // 2. 读入边 (假设是带权有向图)
    // 注意: 如果是重边, 通常保留权值最小的那条
    // int u, v, w;
    // d[u][v] = min(d[u][v], w);

    // 3. 执行算法
    floyd();

    return 0;
}
```

```
}

```

处理负权边 (Dijkstra 的死穴) 这是最本质的区别。Dijkstra 基于贪心思想。它假设“当前找到的最短路径就是最终的最短路径”，一旦确定了一个点的最短距离，就不会再更改。如果图中存在负权边，Dijkstra 的贪心逻辑就会失效，导致答案错误。Floyd 基于动态规划。它穷举了所有的中转可能性。只要图中没有负权环 (Negative Cycle)，Floyd 就能完美处理负权边。场景判断：只要看到题目里边权可能为负数 (且 N 较小)，Dijkstra 直接排除，立刻选 Floyd (或 SPFA/Bellman-Ford)。多源最短路径 vs 单源最短路径 Dijkstra 是单源最短路径。算一次只能求出“起点 S 到其他所有点”的距离。如果你想求“任意两点间距离”，你需要对每个点跑一次 Dijkstra Floyd 是多源最短路径。跑一次，所有点对 (i, j) 的距离都出来了 复杂度对比 (求所有点对距离)：Floyd: $O(N^3)$ N 次 Dijkstra (堆优化): $O(N \cdot M \log N)$ 如果是稀疏图 ($M \approx N$)，Dijkstra 确实更快 ($O(N^2 \log N)$)。如果是稠密图 ($M \approx N^2$)，Dijkstra 会退化成 $O(N^3 \log N)$ ，此时 Floyd 反而更快，而且常数极小。

SPFA 最坏时间复杂度： $O(N \cdot M)$

```
const int N = 100005; // 根据题目要求设定
const int INF = 0x3f3f3f3f;

// 存图: vector<pair<邻居, 权值>>
vector<pair<int, int>> adj[N];
int dist[N]; // 存储起点到各点的最短距离
bool in_queue[N]; // 标记是否在队列中 (避免重复入队)
int cnt[N]; // 记录经过的边数, 用于判断负环
int n, m;

// 返回 true 表示存在负环, false 表示正常求出了最短路径
bool spfa(int s) {
    // 1. 初始化
    memset(dist, 0x3f, sizeof(dist));
    memset(in_queue, 0, sizeof(in_queue));
    memset(cnt, 0, sizeof(cnt));

    dist[s] = 0;
    queue<int> q;
    q.push(s);
    in_queue[s] = true;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        in_queue[u] = false; // 出队后标记为不在队列中

        // 扫描所有出边
        for (auto &edge : adj[u]) {
            int v = edge.first;
            int w = edge.second;

            // 松弛操作: 如果通过 u 到 v 更近
            if (dist[v] > dist[u] + w) {
```

```

        dist[v] = dist[u] + w;
        cnt[v] = cnt[u] + 1; // 记录路径长度

        // 判断负环：如果一条路径经过了 >= n 个点，说明有环
        if (cnt[v] >= n) return true;

        // 如果 v 不在队列中，则入队
        // (这是 SPFA 比 Bellman-Ford 快的原因：只处理更新过的点)
        if (!in_queue[v]) {
            q.push(v);
            in_queue[v] = true;
        }
    }
}
return false; // 没有负环
}

int main() {
    // 读入边...
    // spfa(start_node);
    return 0;
}

```

DP

背包DP

01背包

```

// 必须倒序枚举 j，保证每个物品只被选中一次
for (int i = 1; i <= n; ++i) {
    for (int j = V; j >= w[i]; --j) {
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}

```

完全背包

```

// 必须正序枚举 j，利用当前状态更新后续状态（允许重复选）
for (int i = 1; i <= n; ++i) {
    for (int j = w[i]; j <= V; ++j) {
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}

```

多重背包

- 每种物品有 $s[i]$ 个。
- 思路：将 $s[i]$ 个物品拆分成 $1, 2, 4, \dots, 2^k, \text{remainder}$ 若干个“新物品”，然后跑 0/1 背包。时间复杂度从 $O(V \cdot \sum s[i])$ 降为 $O(V \cdot \sum \log s[i])$ 。

```
// 预处理：拆分物品
struct Item { int w, v; };
vector<Item> items;

for (int i = 1; i <= n; ++i) {
    int count = s[i];
    for (int k = 1; count >= k; k *= 2) {
        items.push_back({w[i] * k, v[i] * k});
        count -= k;
    }
    if (count > 0) {
        items.push_back({w[i] * count, v[i] * count});
    }
}

// 跑 0/1 背包
for (auto& item : items) {
    for (int j = V; j >= item.w; --j) {
        dp[j] = max(dp[j], dp[j - item.w] + item.v);
    }
}
```

分组背包

- 物品被分为 K 组，每组中至多只能选 1 件。

```
// 外层枚举组，中层枚举容量（倒序），内层枚举组内物品
for (int k = 1; k <= groups; ++k) {
    for (int j = V; j >= 0; --j) {
        for (auto& item : group[k]) { // item 包含 {w, v}
            if (j >= item.w) {
                dp[j] = max(dp[j], dp[j - item.w] + item.v);
            }
        }
    }
}
```

二维费用背包

```
// 两个维度都倒序
for (int i = 1; i <= n; ++i) {
```

```

    for (int j = V; j >= w[i]; --j) {
        for (int k = M; k >= m[i]; --k) {
            dp[j][k] = max(dp[j][k], dp[j - w[i]][k - m[i]] + v[i]);
        }
    }
}

```

线性DP

LIS

- **题目：**给个序列nums，求他的最长上升子序列
- $O(n^2)$ 做法：

$$dp[i] = \max(dp[j]) + 1 \quad \text{其中 } 0 \leq j < i, a[i] > a[j]$$

- $O(n \log n)$ 做法：

```

int LIS_nlogn(vector<int>& a) {
    if (a.empty()) return 0;

    // low[i] 存的是长度为 i+1 的子序列的最小结尾
    // 注意：为了方便，low 的下标从 0 开始，所以 low[len-1] 是长度为 len 的结尾
    vector<int> low;

    for (int x : a) {
        // Case 1: 如果 low 为空，或者 x 比 low 最后一个元素大
        if (low.empty() || x > low.back()) {
            low.push_back(x);
        }
        // Case 2: 否则，在 low 里找到第一个 >= x 的位置，替换它
        else {
            // lower_bound 返回迭代器，指向第一个 >= x 的元素
            auto it = lower_bound(low.begin(), low.end(), x);
            *it = x; // 贪心：让该长度的结尾变得更小
        }
    }

    return low.size();
}

```

• 优化思考路径

- 这是一种优化贪心，将dp对象转换成能达到的长度，每一位都代表着如果要达到这个长度我最小要有多大，精辟在更新。相当于转换了dp的两个对象：
- ==最长能有多长->如果要达到这个长度需要最小什么条件==

• ==应用==：

- **Dilworth定理：**最少的下降序列个数就等于整个序列最长上升子序列的长度

- LCS解法->将一个数组的每一个数字跟他的位置映射在一起，相当于开了个桶
- **注意：**求下降序列需要`auto it= upper_bound(all(hei),hsh,greater<int>());`。普通的公式只能算：**排列好后的不下降数列**